

Databázy a vizualizácia

SQL - Príkaz CREATE

doc. Ing. Ladislav Körösi, PhD.

Ústav robotiky a kybernetiky
FEI STU Bratislava

SQL - PRÍKAZ CREATE

Úvod

Príkaz **CREATE** slúži na vytváranie nových databázových objektov.

- databázy,
- tabuľky,
- indexy,
- pohľady,
- funkcie a procedúry.

V tejto časti prednášky sa budeme venovať najmä vytváraniu:

- databáz,
- tabuliek,
- indexov.

Všeobecná forma príkazu:

```
CREATE typ_objektu nazov;
```

SQL - PRÍKAZ CREATE

CREATE DATABASE

Databázu vytvoríme príkazom:

```
CREATE DATABASE nazov_databazy;
```

Príklad:

```
CREATE DATABASE test;
```

Za SQL príkazom sa používa znak ; ktorý oddeľuje jednotlivé SQL príkazy.

SQL - PRÍKAZ CREATE

CREATE DATABASE - IF NOT EXISTS

Ak databáza už existuje, príkaz CREATE skončí chybou.

```
CREATE DATABASE test;
```

Lepšie je použiť:

```
CREATE DATABASE IF NOT EXISTS test;
```

Databáza sa vytvorí len v prípade že ešte neexistuje.

SQL - PRÍKAZ CREATE USE

Pred prácou s tabuľkami je potrebné zvoliť databázu.

```
USE nazov_databazy;
```

Príklad:

```
USE test;
```

Od tohto momentu budú všetky operácie vykonávané vo zvolenej databáze.

SQL - PRÍKAZ CREATE

Znaky a kódovanie

Text uložený v databáze je v skutočnosti uložený ako čísla.

- Každý znak má priradené číslo (tzv. kódový bod).
- Znaková sada (character set) určuje zoznam znakov a ich číselné hodnoty.
- Encoding určuje spôsob, akým sa tieto čísla ukladajú do pamäte v bajtoch.

Príklad:

- znak A má číslo 65,
- znak á má v Unicode číslo 225.

Encoding potom určuje, ako sa toto číslo uloží:

- UTF-8 používa 1 až 4 bajty,
- UTF-16 používa 2 alebo 4 bajty.

Preto môže rovnaký text zaberať rôzny počet bajtov podľa použitého kódovania.

SQL - PRÍKAZ CREATE

Prečo existuje viac typov kódovania

Unicode definuje iba zoznam znakov a ich číselné hodnoty (kódové body). Encoding určuje spôsob, ako sa tieto čísla uložia do pamäte. Rovnaký znak je možné uložiť rôznymi spôsobmi:

- UTF-8 používa pre znak 1 až 4 bajty,
- UTF-16 používa 2 alebo 4 bajty,
- UTF-32 používa vždy 4 bajty.

Rozdiel je v kompromisoch medzi úložným priestorom a jednoduchosťou spracovania.

- UTF-8 šetrí miesto pre texty v angličtine alebo európskych jazykoch, pretože väčšina znakov má 1 alebo 2 bajty
- UTF-16 je efektívnejšie pre jazyky s veľkým počtom znakov napr. čínština alebo japončina
- UTF-32 je najjednoduchšie na spracovanie, pretože každý znak má rovnakú dĺžku, ale zaberá najviac miesta

Výber encodingu je teda kompromis medzi veľkosťou dát a jednoduchosťou spracovania.

SQL - PRÍKAZ CREATE

Character set a Collation

Pri vytváraní databázy môžeme určiť spôsob ukladania a porovnávania textu.

- **Character set** určuje aké znaky je možné ukladať (napr. latin1, utf8, utf8mb4).
- **Collation** určuje pravidlá porovnávania a triedenia textu.

Character set určuje množinu znakov, napr.:

- latin1 – základná znaková sada
- utf8 alebo utf8mb4 – podpora väčšiny jazykov sveta

Collation určuje ako sa text porovnáva. Napríklad:

- či sa rozlišujú veľké a malé písmená ($A = a$ alebo $A \neq a$)
- či sa rozlišuje diakritika ($a = \acute{a}$ alebo $a \neq \acute{a}$)
- ako sa text triedi pri ORDER BY

Napríklad collation utf8_general_ci je *case insensitive*, teda $A = a$.

SQL - PRÍKAZ CREATE

CREATE DATABASE – Character set

Pri vytváraní databázy môžeme určiť znakové kódovanie.

```
CREATE DATABASE nazov_databazy  
DEFAULT CHARACTER SET charset_name  
COLLATE collation_name;
```

Príklad:

```
CREATE DATABASE test  
DEFAULT CHARACTER SET latin1  
COLLATE latin1_swedish_ci;
```

Tieto nastavenia sa použijú ako predvolené pre tabuľky.

SQL - PRÍKAZ CREATE

CREATE TABLE

Tabuľku vytvoríme príkazom:

```
CREATE TABLE nazov_tabulky (  
    stlpec1 datatype,  
    stlpec2 datatype,  
    stlpec3 datatype  
);
```

Tabuľka obsahuje:

- názov tabuľky,
- názvy stĺpcov,
- dátové typy stĺpcov.

SQL - PRÍKAZ CREATE

CREATE TABLE - príklad

```
CREATE TABLE filmy (  
    id INT,  
    nazov VARCHAR(50),  
    rok YEAR,  
    imdb DECIMAL(3,1)  
);
```

Každý stĺpec má názov a dátový typ.

SQL - PRÍKAZ CREATE

Character set a Collation v tabuľke

Ak pri vytváraní tabuľky neuvedieme znakové nastavenia, tabuľka preberá **character set** a **collation** z databázy.

Niekedy však potrebujeme použiť iné nastavenia. V takom prípade ich môžeme definovať priamo v príkaze CREATE TABLE.

- pre celú tabuľku,
- alebo pre jednotlivé textové stĺpce.

Napríklad môžeme určiť:

- predvolený character set tabuľky,
- collation tabuľky,
- alebo character set a collation konkrétneho stĺpca.

Takto môžeme napríklad používať inú collation pre triedenie textu alebo inú znakovú sadu pre konkrétny stĺpec. Používa sa to napríklad pri databázach obsahujúcich viac jazykov, pri špecifických pravidlách triedenia textu alebo pri optimalizácii veľkosti uložených dát.

SQL - PRÍKAZ CREATE

CREATE TABLE – Character set

Nastavenia môžeme definovať aj priamo pri vytváraní tabuľky.

```
CREATE TABLE filmy (  
    ID INT(11),  
    NAZOV CHAR(50)  
        CHARACTER SET utf8  
        COLLATE utf8_slovak_ci,  
    ROK YEAR(4),  
    IMDB DECIMAL(2,1)  
)  
ENGINE = InnoDB  
DEFAULT CHARSET = latin1;
```

Character set a collation môžeme nastaviť pre celú tabuľku alebo pre jednotlivé textové stĺpce.

SQL - PRÍKAZ CREATE

Storage engine

MySQL podporuje viacero tzv. **storage engine**. Storage engine určuje spôsob, akým sú dáta tabuľky fyzicky uložené a spravované v databáze.

Každý engine má rôzne vlastnosti, napríklad:

- podporu transakcií,
- spôsob uzamykania dát,
- výkon pri čítaní alebo zápise,
- spôsob ukladania dát na disk alebo do pamäte.

V moderných verziách MySQL je predvolený engine **InnoDB**. Predvolený storage engine je nastavený na úrovni databázového servera (napr. premenná `default_storage_engine`). Tabuľky toto nastavenie automaticky preberajú, pokiaľ engine pri `CREATE TABLE` neurčíme explicitne.

SQL - PRÍKAZ CREATE

Príklady storage engine

Niektoré často používané storage engine:

- **InnoDB** - Najpoužívanejší engine v MySQL. Podporuje transakcie a foreign keys.
- **Memory** - Dáta sú uložené v RAM pamäti, preto je prístup veľmi rýchly. Používa sa napríklad pre dočasné alebo pomocné tabuľky.
- **NDB Cluster** - Engine používaný v distribuovaných databázových clusteroch, kde je dôležitá vysoká dostupnosť.

Zoznam podporovaných engine môžeme zistiť príkazom:

```
SHOW ENGINES;
```

SQL - PRÍKAZ CREATE

Obmedzenia (Constraints)

SQL **constraints** slúžia na definovanie pravidiel pre údaje v tabuľke.

Constraint určuje aké hodnoty je možné do stĺpca uložiť.

Ak sa pokúsime vložiť údaje ktoré porušujú definované pravidlo, databáza operáciu odmietne.

Constraints môžeme definovať:

- pri vytváraní tabuľky (CREATE TABLE),
- alebo dodatočne pomocou príkazu ALTER TABLE.

Cieľom constraints je zabezpečiť správnosť a integritu dát.

SQL - PRÍKAZ CREATE

Typy SQL constraints

Najčastejšie používané constraints v SQL sú:

- **NOT NULL** - Stĺpec nesmie obsahovať NULL.
- **UNIQUE** - Každá hodnota v stĺpci musí byť jedinečná.
- **PRIMARY KEY** - Jednoznačný identifikátor riadku v tabuľke.
- **FOREIGN KEY** - Zabezpečuje referenčnú integritu medzi tabuľkami.
- **CHECK** - Kontroluje či hodnota spĺňa definovanú podmienku.
- **DEFAULT** - Definuje predvolenú hodnotu stĺpca.

Tieto pravidlá pomáhajú zabezpečiť konzistentnosť databázy.

SQL - CONSTRAINT

NOT NULL

Constraint **NOT NULL** zabezpečuje, že stĺpec musí obsahovať hodnotu.

```
CREATE TABLE filmy (  
    id INT,  
    nazov VARCHAR(50) NOT NULL,  
    rok YEAR,  
    imdb DECIMAL(3,1)  
);
```

Stĺpec nazov **nemôže** byť prázdny.

Pri pokuse vložiť záznam bez názvu filmu databáza operáciu odmietne.

SQL - CONSTRAINT

UNIQUE

Constraint **UNIQUE** zabezpečuje, že hodnoty v stĺpci sa nemôžu opakovať.

```
CREATE TABLE filmy (  
    id INT,  
    nazov VARCHAR(50) UNIQUE,  
    rok YEAR,  
    imdb DECIMAL(3,1)  
);
```

Každý film musí mať jedinečný názov.

Ak sa pokúsime vložiť film s rovnakým názvom, databáza operáciu odmietne. V praxi to však nemusí byť vhodné riešenie, pretože rôzne filmy môžu mať rovnaký názov (napr. remaky alebo filmy z rôznych rokov).

SQL - CONSTRAINT

FOREIGN KEY

Constraint **FOREIGN KEY** prepája tabuľky a zabezpečuje referenčnú integritu medzi nimi.

Predpokladajme, že existuje tabuľka `filmy`, kde stĺpec `id` je **PRIMARY KEY**.

```
CREATE TABLE hodnotenie (  
    id INT PRIMARY KEY,  
    film_id INT,  
    hodnotenie INT,  
    FOREIGN KEY (film_id)  
        REFERENCES filmy(id)  
);
```

Stĺpec `film_id` musí obsahovať hodnotu, ktorá existuje v stĺpci `id` tabuľky `filmy`. Jeden film tak môže mať viac hodnotení od rôznych používateľov alebo z rôznych zdrojov.

SQL - CONSTRAINT

CHECK

Constraint **CHECK** zabezpečuje, že hodnota spĺňa definovanú podmienku.

```
CREATE TABLE filmy (  
    id INT PRIMARY KEY,  
    nazov VARCHAR(50),  
    rok YEAR,  
    imdb DECIMAL(3,1),  
    CHECK (imdb >= 0 AND imdb <= 10)  
);
```

Hodnotenie filmu musí byť v rozsahu 0 až 10.

SQL - CONSTRAINT

DEFAULT

Constraint **DEFAULT** nastaví predvolenú hodnotu stĺpca.

```
CREATE TABLE filmy (  
    id INT PRIMARY KEY,  
    nazov VARCHAR(50),  
    rok YEAR,  
    imdb DECIMAL(3,1) DEFAULT 0  
);
```

Ak pri vkladaní hodnotenie neuvedieme, databáza automaticky použije hodnotu 0.

SQL - PRÍKAZ CREATE

AUTO_INCREMENT

AUTO_INCREMENT umožňuje automaticky generovať jedinečnú hodnotu pre identifikačný stĺpec tabuľky.

- najčastejšie sa používa pre primárny kľúč,
- databáza automaticky priradí ďalšiu hodnotu pri vložení nového riadku,
- hodnoty sa postupne zvyšujú (1, 2, 3, ...).

Príklad:

```
CREATE TABLE filmy (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nazov VARCHAR(50),  
    rok YEAR  
);
```

Index je pomocná dátová štruktúra, ktorá umožňuje databáze rýchlejšie vyhľadávať údaje v tabuľke.

Bez indexu musí databáza pri vyhľadávaní prečítať celú tabuľku riadok po riadku.

S indexom databáza vie nájsť požadované údaje podobne ako v registri alebo v obsahu knihy.

Indexy:

- zrýchľujú vyhľadávanie údajov v tabuľke (napr. pri príkazoch SELECT),
- môžu spomaliť operácie INSERT, UPDATE a DELETE, pretože sa musí aktualizovať aj index.

Indexy by sa mali vytvárať najmä na stĺpcoch, ktoré sa často používajú pri vyhľadávaní.

SQL - PRÍKAZ CREATE

CREATE INDEX

Index vytvoríme príkazom:

```
CREATE INDEX index_name  
ON table_name(column_name);
```

Príklad:

```
CREATE INDEX idx_nazov  
ON filmy(nazov);
```

Index môžeme vytvoriť aj nad viacerými stĺpcami:

```
CREATE INDEX idx_film  
ON filmy(nazov, rok);
```

Pri použití **FOREIGN KEY** musí byť daný stĺpec indexovaný, aby databáza vedela rýchlo kontrolovať existenciu príslušných záznamov v druhej tabuľke.